

Python Notepad Tracker Assignment

Intern: Sumit Dhar

Date: 25th September, 2025

Objective:

To create a simple, user-friendly **browser-based notepad** that allows users to **open, edit, and auto-save files or create new ones** using **manual paths**, while keeping the code modular, reusable, and clean.

Step 1: Project Folder Structure

```
notepad_tracker/  
├── backend/  
│   ├── routes.py  
│   ├── utils/  
│   │   ├── file_utils.py  
│   │   └── git_utils.py  
│   └── config.py  
├── frontend/  
│   ├── static/  
│   │   ├── css/styles.css  
│   │   └── js/autosave.js  
│   └── templates/index.html  
├── default_storage/  
│   ├── notes1.txt  
│   ├── notes2.txt  
│   ├── notes3.txt  
│   ├── .  
│   ├── .  
│   └── .  
└── app.py
```

- Initialized **Flask** app and registered '**routes.py**' blueprint.
-

Step 2: Text Editor UI & File Handling

Goal: Let users provide a file path and start editing.

- Added a **<textarea>** and **<input>** field in '**index.html**' to enter a file path and press 'Enter'.

- If file exists: contents are loaded.

Notepad Tracker

`F:\Internship\Python\notepad_tracker\default_storage\note1.txt`

Press Enter or use Browse buttons

`I am creating first file mentioning a new file name directly in file path input.
This is the second time I am updating this file.|`

- If path is a directory: user is prompted to enter filename.

Notepad Tracker

`F:\Internship\Python\notepad_tracker\default_storage\`

Press Enter or use Browse buttons

127.0.0.1:5000 says

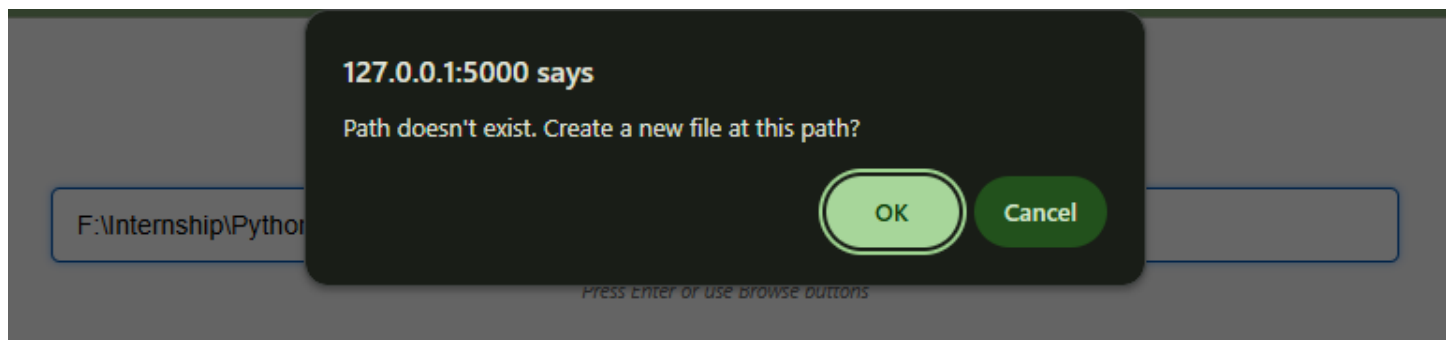
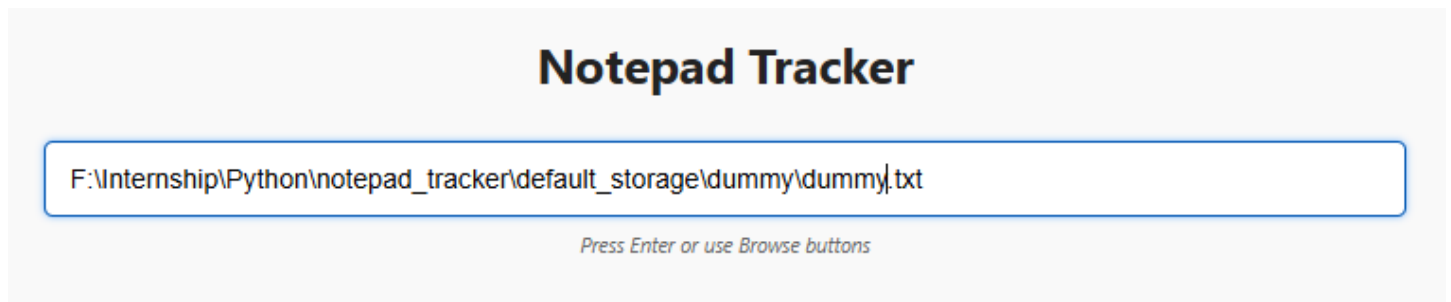
This is a directory. Enter filename to create inside it:

`note2.txt`

OKCancel

`F:\Internship\Python\notepad_tracker\default_storage\`

- If path doesn't exist: user is asked to create new file.



- Autosave implemented with a **5-second debounce**.

```
127.0.0.1 - - [25/Sep/2025 17:03:02] "GET / HTTP/1.1" 200 -  
127.0.0.1 - - [25/Sep/2025 17:03:02] "GET /static/css/styles.css HTTP/1.1" 304 -  
127.0.0.1 - - [25/Sep/2025 17:03:02] "GET /static/js/autosave.js HTTP/1.1" 200 -  
127.0.0.1 - - [25/Sep/2025 17:03:14] "POST /check_path HTTP/1.1" 200 -  
127.0.0.1 - - [25/Sep/2025 17:03:14] "POST /load HTTP/1.1" 200 -  
127.0.0.1 - - [25/Sep/2025 17:03:35] "POST /save HTTP/1.1" 200 -  
□
```

Step 3: Saving Content to File

Goal: Autosave content from editor.

- Wrote `save_file(filepath, content)` in `'file_utils.py'`.
- Autosave triggered only if file is confirmed and content is not empty.
- Backend saves the file using: **POST /save**.

```
# backend/utils/file_utils.py
import os

def save_file(filepath, content):
    dir_path = os.path.dirname(filepath)
    os.makedirs(dir_path, exist_ok=True)

    with open(filepath, 'w', encoding='utf-8') as f:
        f.write(content)
```

Step 4: Git Commit Integration

Goal: Track file changes using Git.

- Added `commit_file(filepath)` in `'git_utils.py'`:
 - Adds and commits file using **subprocess** and **Git CLI**.
- Automatically triggered on each save.

```
# backend/utils/git_utils.py
import os
import git
from backend.config import AUTO_COMMIT_MESSAGE

def commit_file(filepath):
    repo_path = os.path.dirname(filepath)

    try:
        repo = git.Repo(repo_path, search_parent_directories=True)
    except git.exc.InvalidGitRepositoryError:
        repo = git.Repo.init(repo_path)

    rel_path = os.path.relpath(filepath, repo.working_tree_dir)
    repo.index.add([rel_path])
    repo.index.commit(AUTO_COMMIT_MESSAGE)
```

Step 5: Toast Notification System

Goal: Show non-intrusive messages for save success, errors, etc.

- Created a **showToast(message, type)** helper in JS.
- Styled using **CSS** with fade-out and click-to-dismiss.
- Integrated into all key user interactions (save, load, errors, etc).

```
// Toast notification helper
function showToast(message, type = 'info') {
  const container = document.getElementById('toast-container');
  const toast = document.createElement('div');
  toast.classList.add('toast', type);
  toast.textContent = message;

  // Remove toast on click
  toast.addEventListener('click', () => {
    container.removeChild(toast);
  });

  container.appendChild(toast);

  // Auto remove after 5 seconds (matches fadeOut animation)
  setTimeout(() => {
    if (container.contains(toast)) {
      container.removeChild(toast);
    }
  }, 5000);
}
```

- Without specifying path and pressing enter, error toast appears.

Notepad Tracker

Please enter a valid path.

Enter full file path to start editing

Press Enter or use Browse buttons

- After successfully file auto-saved, success toast appears.

Notepad Tracker

File saved successfully!

F:\Internship\Python\notepad_tracker\default_storage\note1.txt

Press Enter or use Browse buttons

I am creating first file mentioning a new file name directly in file path input.
This is the second time I am updating this file.
This is the third time I am updating this file.

Summary:

We built a **browser-based notepad** that lets users **open**, **edit**, and **auto-save** files by entering paths. The app checks if a path is a **file**, **folder**, or new, then loads or creates files accordingly. It supports helpful toast notifications for feedback. The code was refactored to use reusable helper functions for cleaner, modular logic, improving maintainability.

- Implemented backend routes to check paths, list directories, load files, and save content.
- Used async JS functions with fetch API for smooth communication between frontend and backend.
- Improved UX with debounced auto-saving.
- Refactored repeated logic into reusable helper functions for cleaner, maintainable code.

This results in a simple, user-friendly text editor with robust file handling and smooth UX.